

Lab 3

In this lab, you'll practice using lists and iterators.

Finding frequent phrases

One way to measure public interest in a topic is to look at what people are searching for on popular search engines (e.g. <https://trends.google.com/trends/>). The data is basically gathered by counting the number of times each query is entered and reporting on the most common. (To do this well, you'd also want to look for non-identical related queries, but that's beyond the scope of our lab.) We're going to simulate this by reading lines from a text file and ordering them by frequency (most common line first, then next most common, etc).

Begin by downloading the given code (`lab3given.zip`), unzipping it, and then opening the project by double clicking `lab3.code-workspace`. This project contains a single file of code, `QueryCount.java`, which is basically empty. Replace its contents with our starter code:

```
import java.util.*;
import java.io.*;

public class QueryCount {
    public static void main(String[] args) {
        Scanner input = null;
        try {
            input = new Scanner(new File("input.txt"));
        } catch (FileNotFoundException ex) {
            System.err.println("Couldn't find input file");
            System.exit(1);
        }

        while(input.hasNextLine()) {
            String line = input.nextLine();

        }
    }
}
```

This program opens a file called `input.txt` and reads its contents one line at a time. Get that to compile. There is already a file `input.txt`, but it's empty. Give it a couple of lines of text, including some duplicates. Then you should be able to run the program, though it doesn't do anything yet.

To give the program some visible behavior, declare an attribute of `QueryCount` that is an `ArrayList<String>` to hold the text that you've read. Create a constructor to initialize this attribute, an `add` method of `QueryCount` that takes a string and adds it to the list, and a `print` method that prints a message about how many queries are in the list. (Remember that you can get the number of elements in a list using its `size` method.) Then, modify `main` so that it creates a `QueryCount` object, adds each line it reads to that object, and calls the `print` method after the input-reading loop.

Now you should have a program that correctly reports the number of input lines. Next, make it print the actual lines using an iterator by adding the following to the `print` method (using your variable names):

```
ListIterator<String> it = list.listIterator();
```

```

while(it.hasNext()) {
    System.out.println(it.next());
}

```

This gives you a program that counts the lines of input and then echos them.

In order to find frequent queries, we need to consolidate duplicate values into a single list element instead of creating a new element for each one. Create a class **Query** with a string attribute (the query) and an integer attribute (the number of times it has been seen). For nice printing, give **Query** a method to convert itself into a string (using your attribute names if they're different from these):

```

public String toString() {
    return query + " " + num;
}

```

Then modify the list in **QueryCount** to be of **Query** objects instead of strings. Also modify the **print** method so its iterator goes over **Query** objects instead of strings. Finally, modify the **add** method so that it uses an iterator to traverse the list looking to see if the query it is adding is already in the list. If it is, then increment the counter for that query by one. (Note that **next** returns a reference to the **Query** object that you can use to modify the object's counter and that you don't need to actually change the list or add the changed object.) If the query isn't in the list, then add a new **Query** object (with count 1) for it to the end of the list.

After all that, you should have a program that prints each distinct query only once, but includes a frequency count for each of them. Try a couple of different inputs to make sure that this works.

For our final improvement, let's make the list store queries in order of their frequency. (That way, the most common queries are at the front and can be accessed easily.) To do this, I want you to change the **add** method so that it moves a query earlier in the list if it is more common than the previous query in the list. You can do this entirely using methods of **listIterator**; remember that this class provides **hasPrevious** and **previous** methods for going backwards in the list and **add** and **remove** methods for changing the list at the current location; see the notes below. You'll want to remove the query, find the right place for it, and then add it back to the list.

Be sure to test your method, including on a case when a query needs to move more than one place earlier in the list. (For example, it could be the last in a long sequence of queries with the same frequency. Then it is repeated and needs to move before all the others.)

Information about iterators

A helpful mental model for list iterators: at all times, the iterator is *between* elements rather than on them. Specifically, it is positioned after the element it will return on a call to **previous** and before the one it will return on a call to **next**. The **remove** method will remove whichever of these elements was most recently returned (by **next** or **previous**). The **add** method will add in the iterator's current position.